

# Autonomous Drone Racing

---

*Session 2: Project Intro and Next Steps*

22.04.2026

Marcel Rath

*Chair of Safety, Performance and Reliability for Learning Systems  
School of Computation, Information and Technology*





# What We Cover Today

---

1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]

# Installing the Drone Racing Repository (Ubuntu)



- Open a terminal and install [pixi](https://pixi.sh/) (close after installation)

```
curl -fsSL https://pixi.sh/install.sh | sh
```

- Fork the [lsy\\_drone\\_racing repository](https://github.com/ratheron/lsy_drone_racing)  
(keep the name identical and add your team member)

- Clone the repository and open the folder

```
git clone https://github.com/<username>/lsy_drone_racing.git
```

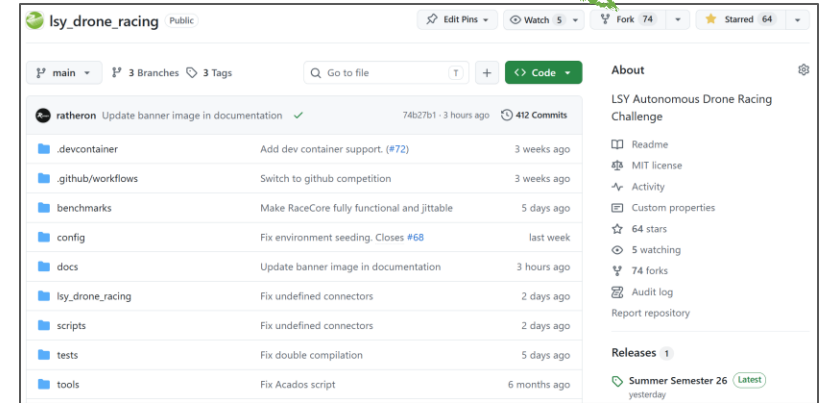
```
cd lsy_drone_racing
```

- Install the pixi environment and run the tests

```
pixi install
```

```
pixi run tests
```

⇒ For other systems, check out the docs



# Documentation



- You can find all the information in the [docs](#)
- Includes setup, course info, and the Python API
- Let us know if something is missing!

The screenshot shows the LSY Drone Racing documentation website. The header includes a search bar, navigation icons, and a 'Contents' sidebar. The main content area features a large image of a drone racing track and the title 'LSY Drone Racing'. Below the title is a welcome message and a 'Project Overview' section with a list of key features. The left sidebar contains a 'Getting Started' section with links to 'The Drone Racing Project' and 'Installation and Setup', a 'The Challenge' section with links to 'Overview', 'Simulation', 'Deployment', and 'Online Competition', and a 'Python API' section with links to 'lsy\_drone\_racing.control', 'lsy\_drone\_racing.envs', 'lsy\_drone\_racing.utils', and 'lsy\_drone\_racing.ros'.

## LSY Drone Racing

Welcome to the LSY Drone Racing documentation! This project provides a simulator and code for deployment on real robot hardware for autonomous drone racing.

### Project Overview #

LSY Drone Racing is designed to help you develop and test autonomous drone racing algorithms. Whether you're a beginner or an experienced drone enthusiast, this documentation will guide you through the process of setting up your environment, understanding the challenge, and utilizing the provided Python API to create high-performance racing drones.

#### Key Features:

1. **Simulation Environment:** Test your algorithms in a realistic virtual environment before deploying to real drones.
2. **Deployment Tools:** Transition from simulation to real-world racing with interfaces compatible with the simulation environment.
3. **Online Competition:** Participate in virtual drone racing competitions to benchmark your algorithms against others.
4. **Comprehensive Python API:** Access a comprehensive documentation of the Python API to get a deeper understanding of how the simulation and deployment environments work under the hood.

#### Getting Started

If you're new to the project, start with the [Getting Started](#) section. This will help you set up your environment



# Challenge Levels

- Level 0:
  - Drones fly in simulation with perfect knowledge
  - No randomness in drone properties, gates, obstacles, etc.
  - Slight action and dynamics disturbances
- Level 1:
  - Drones have a slightly perturbed mass & inertia
- Level 2:
  - Gate position and obstacles are randomized (deviation from nominal position)
  - Exact location revealed when closer than a threshold
- Level 3:
  - Entire track is randomized and unknown before runtime (drone, gates, obstacles)

Simulation

- Deploy: The real thing!
  - We deploy your code in our lab
  - Your algorithm's performance is measured with real drones

Real Drones

# Configs

---



- The [config files](#) define the environment
- Do **not** change the config files, except:
  - The path to the controller (`controller.file`)
  - The environment action space (`env.control_mode`)
- Have a look at the settings to understand the different levels
- It's perfectly valid to use the information in your controller



# Action Interfaces

---

- You can choose between two interfaces:
  - Desired position, velocity, acceleration, yaw, rpy-rates (13D)
  - Attitude (as rpy angles) + collective thrust (4D)
- Choose your interface depending on your control strategy
- Change the environment control mode in the config!
- Both interfaces can be deployed on the real drones



# Creating Your Own Controller

---

- Implement a controller following the [base controller](#)
- Do **not** change the function signatures
- You can find some example controllers in the [control module](#)
- Put your controller into the same module
- You are free to use any control strategy that works





# Reporting bugs and problems

---

- If you encounter bugs or are missing features, please let us know
- Check if the issue is already known!
- Open an issue on GitHub and describe what's wrong or missing
  - Be clear and provide all necessary information to understand your issue
  - If possible, add a minimal example
- The issue tracker is [here](#)



# What We Cover Today

---

1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]



# What We Cover Today

---

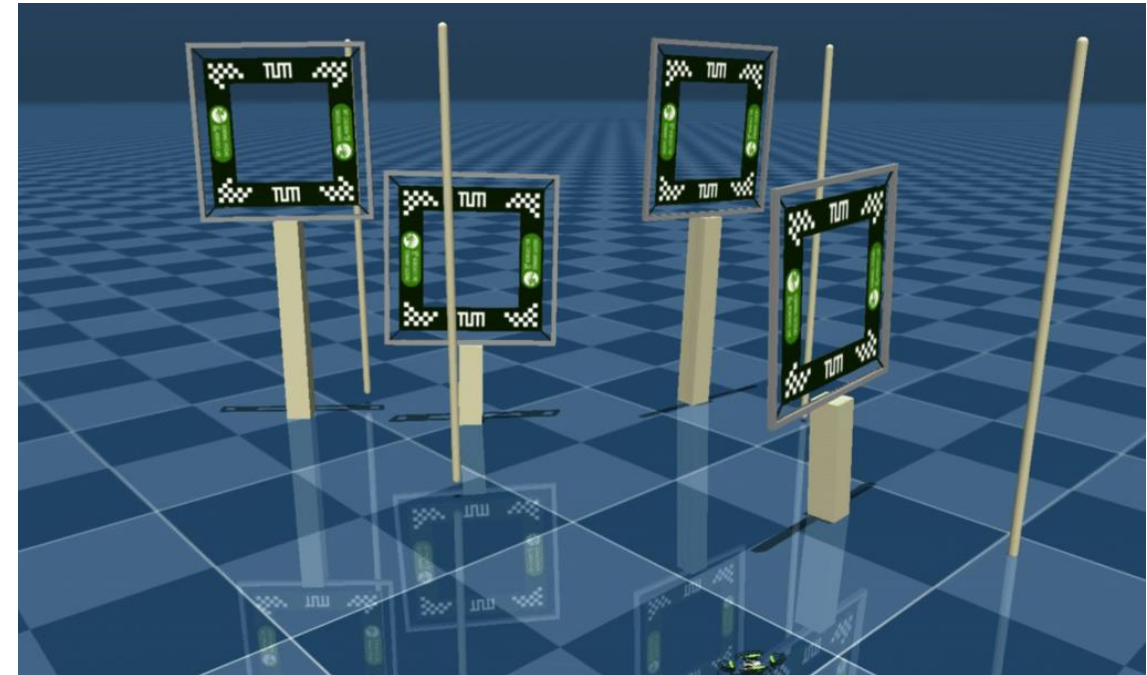
1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]

# The Competition



- Goal: Pass level 2 with at least 50% success rate (20 runs)
- Difficulties:
  - Action and state noise
  - Randomized drone parameters (mass, inertia)
  - Randomized gates and obstacles (exact location revealed when in sensor range)

3.92s lap time & 70.1% success rate





## Developing a Controller for the Competition

---

1. Add your controller to `config/level2.toml`
  - You may start with a solution working on level 0
  - Don't to forget to adjust the `control_mode` to your control interface
2. Develop your controller by testing it in simulation running (in pixi env)  
`python scripts/sim.py --config level2.toml`
3. When you are confident, you may run the evaluation script (20 test runs)  
`pixi run evaluate` or `python scripts/evaluation.py`
4. A run counts as successful when you can beat level 2 with  $\geq 50\%$  success rate



## Participating in the Online Competition

---

1. Register your team → your **main repository** will be added to the whitelist
2. Activate actions in your main repository
3. Add the `DISPATCH_TOKEN` (given on moodle) to your repository secrets  
→ Each push to your **main** branch triggers an evaluation on our central repo

⇒ If a new solution is **successful** and **faster** than the current one, the leaderboard is updated

⇒ Since only pushes to main trigger the evaluation, please use **other branches** of your fork for development to not spam the evaluation (only submit when ready)

ratheron / lsy\_drone\_racing

Code Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

Secrets and variables

Actions

Codespaces

Dependabot

## Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets Variables

### Environment secrets

This environment has no secrets.

Manage environment secrets

### Repository secrets

This repository has no secrets.

New repository secret

3.

2.

1.

amacati / lsy\_drone\_racing

Code Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

## Workflows aren't being run on this forked repository

Because this repository contained workflow files when it was forked, we have disabled them from running on this fork. Make sure you understand the configured workflows and their expected usage before enabling Actions on this repository.

I understand my workflows, go ahead and enable them

[View the workflows directory](#)

## Actions secrets / New secret

Name \*

DISPATCH\_TOKEN

Secret \*

<insert token from moodle>

Add secret

5.



## Participating in the Online Competition – Notes

- Do not submit manually, let GitHub do it for you
- Doing changes beyond what we have discussed will make the evaluation fail on our side
- You might need to add your extra dependencies to the `pyproject.toml`
- The online competition is **not** graded
- It is an opportunity for you to check how good your solution is compared to others

| Rank | Team                         | Time   |
|------|------------------------------|--------|
| 1    | VIERFACHTURBOLADENERFRESSACK | 4.290  |
| 2    | Carl Schiedat                | 4.836  |
| 3    | Felix Durchdewald            | 5.664  |
| 4    | Keno and Simon               | 6.596  |
| 5    | Max Christof                 | 6.740  |
| 6    | Julius B                     | 7.040  |
| 7    | Antonio Steiger              | 7.070  |
| 8    | Ahmed                        | 10.288 |





# What We Cover Today

---

1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]



# What We Cover Today

---

1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]



# Final Report and Presentation

---

## Report

- 3 double column pages per person (6 in total, excluding references)
- Summarize your algorithms, implementation and results
- Produce relevant figures and diagrams

## Presentation

- 5 minutes per person
- Explain your approach and show results (plots & video)
- Follow-up Q&A session



## Performance and implementation: 40%

- No collisions, all gates passed, robustness against disturbances and noise
- Fast track times
- Well-documented, clean code (see lecture 4)

## Written report: 30%

- Clear and logical structure, covers all central issues
- Stays in page limit

## Final presentation: 30%

- Clear and logical structure, covers all central issues, tailored to audience
- Stays in time limit



# Overall Course Expectations

---

- No requirement for novelty
- Using open-source code is encouraged
  - Check the exact license though!
- Requirement for a working solution
  - Early integration is key!
  - Adjustment of goals may be necessary to achieve a working solution by the end of the term



# What We Cover Today

---

1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]



# What We Cover Today

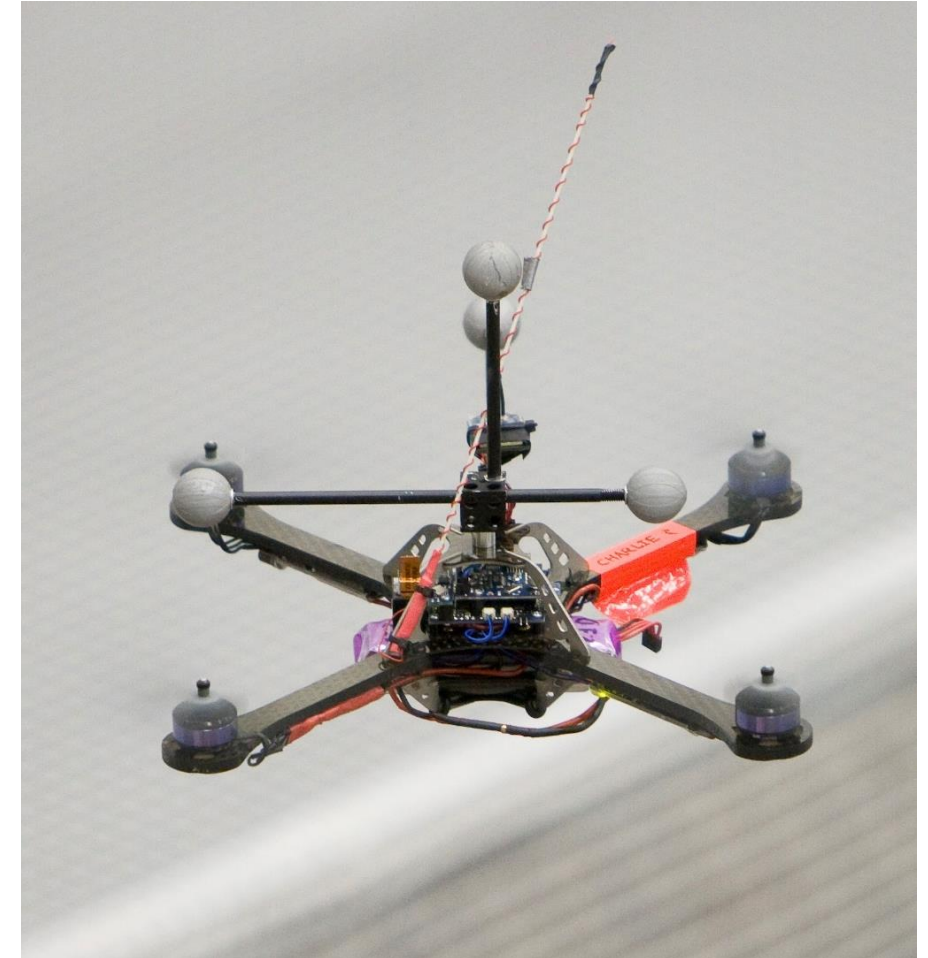
---

1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]



# Why Quadrotors?

1. Rigid frame
  2. Four independently controlled motors
  3. Move by changing the motor speeds
  4. Vertical take-off and landing (VTOL) capable
- Mechanically simple
  - Highly maneuverable

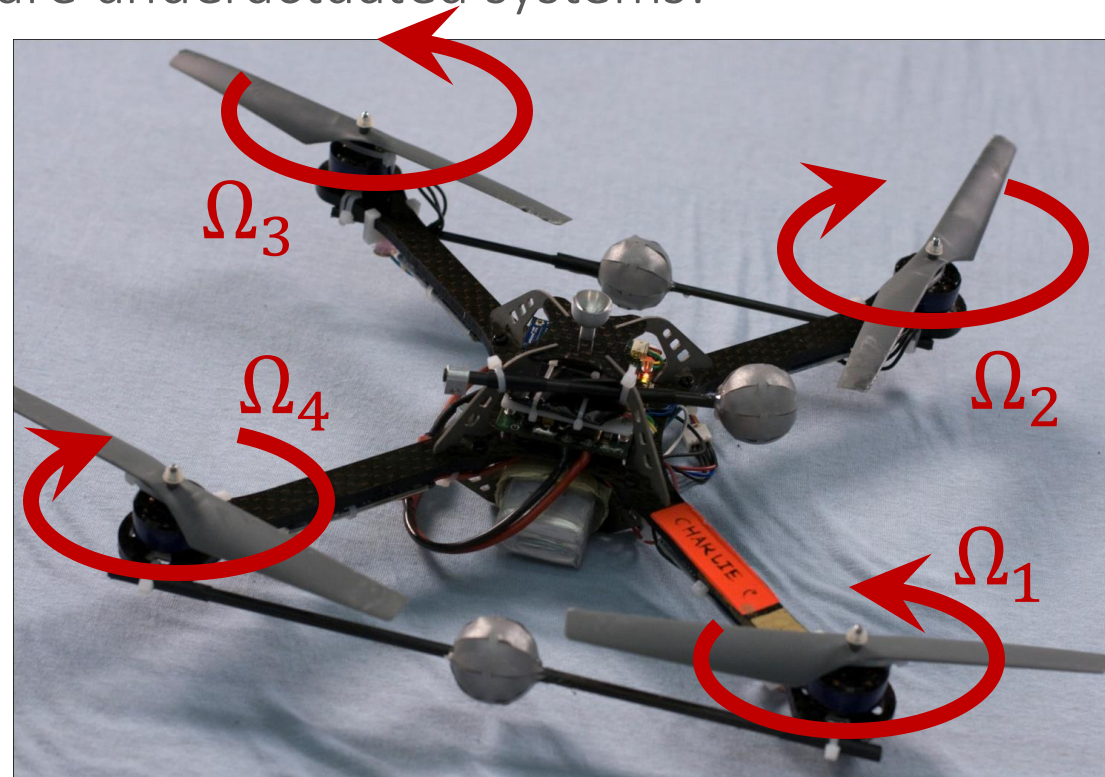




# Quadrotor Vehicle



- Vary the speeds of the rotors to control the position and orientation (attitude) of the robot
- Note: Quadrotors are underactuated systems!



# Drone Hover



Green arrows represent the **forces** produced by the motors, which is roughly proportional to the **squared rotor speed**

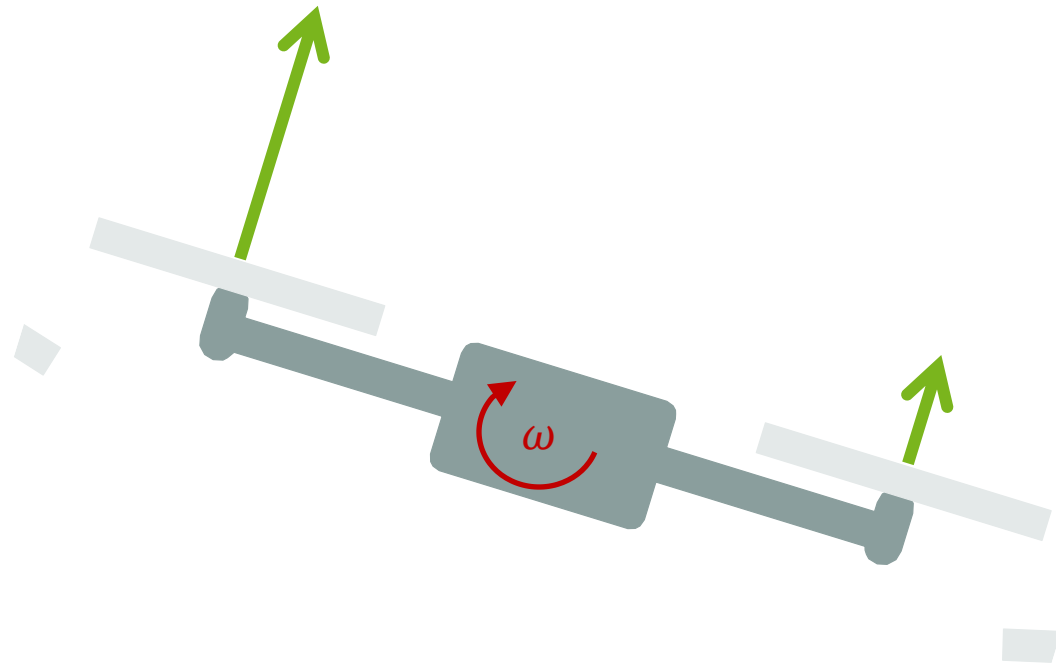


# Drone Rotation: Roll & Pitch

---



# Drone Rotation: Roll & Pitch





# Drone Acrobatics (Flips up to 1600 deg / sec)

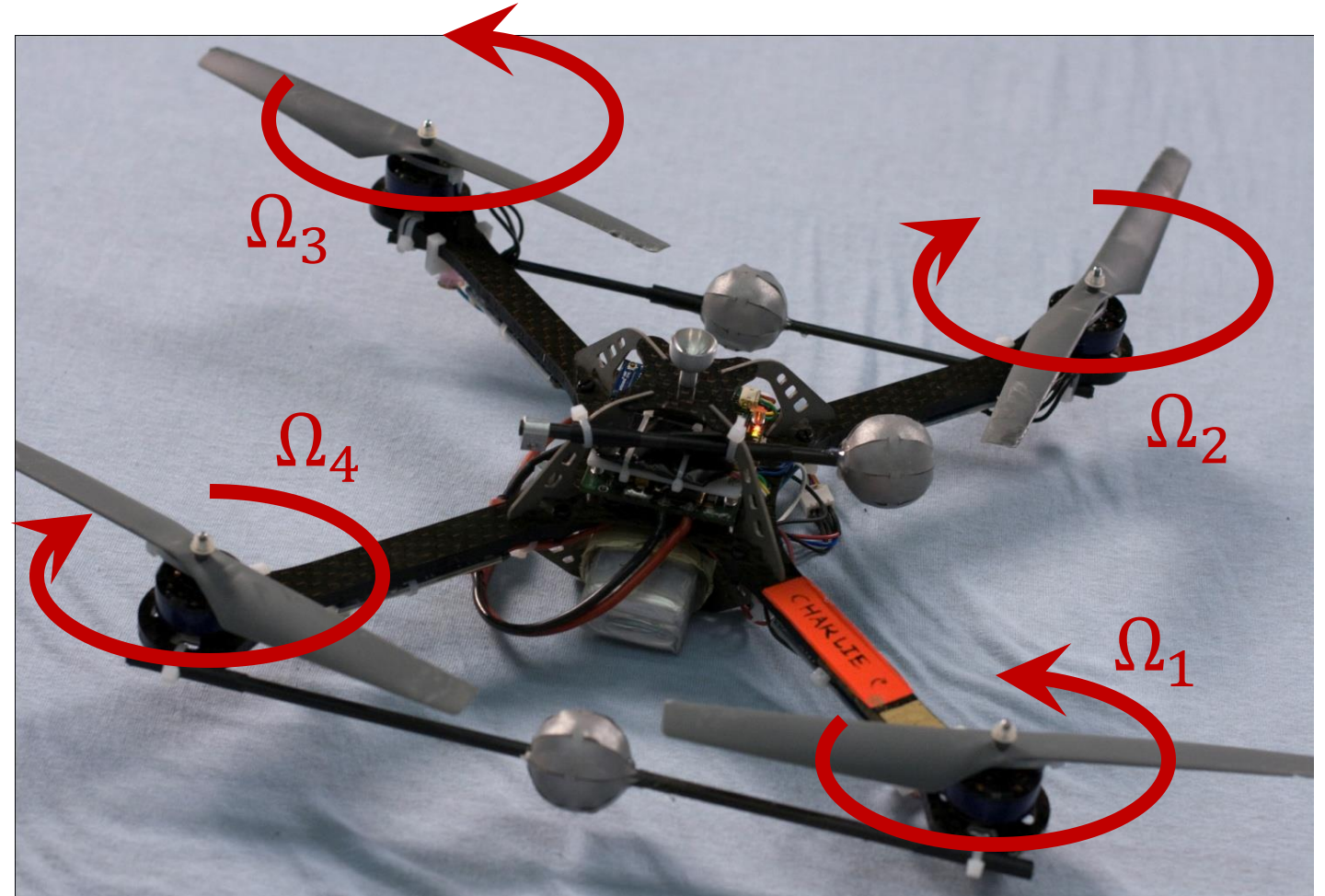




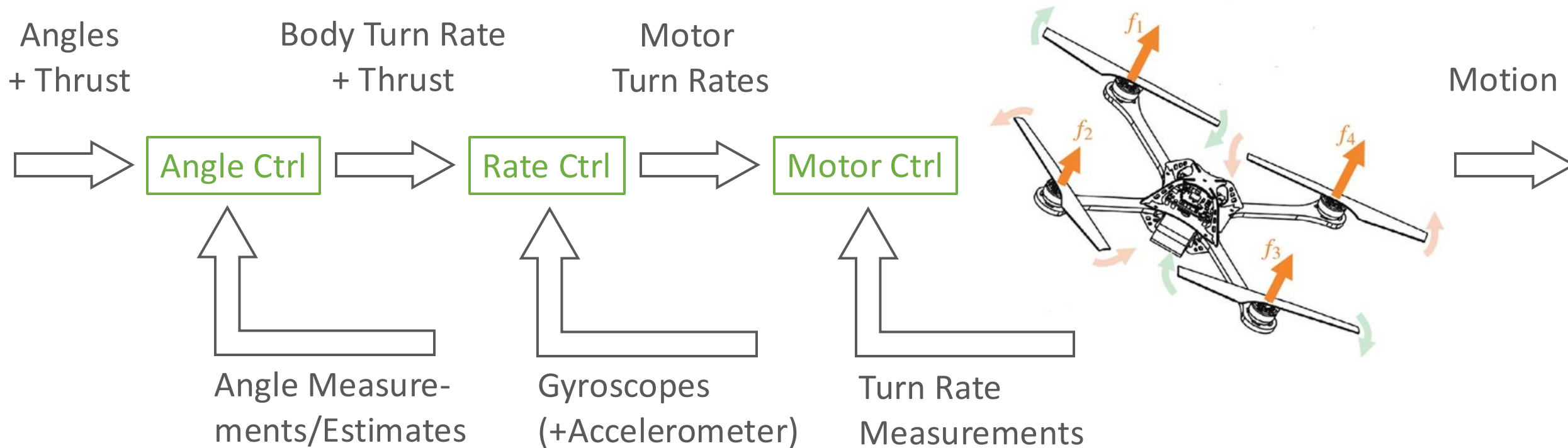
## Drone Rotation: Yaw

How to steer yaw?

- Propellers create
  - Thrust force
  - Drag torque
- Drag torques cancel in hover
- Increase/Decrease opposite motor speeds



# Quadrotor Control



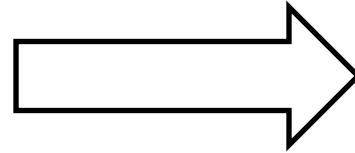
Control allows us to focus on the **high-level task**



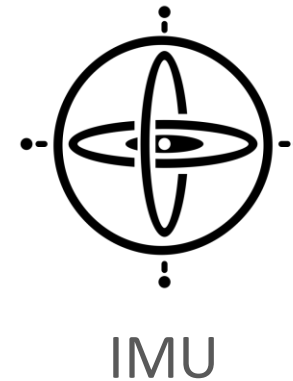
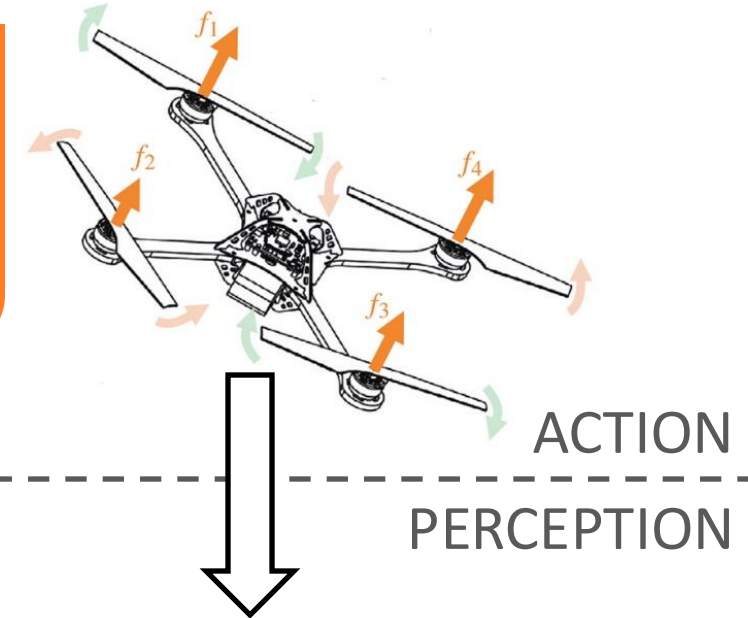
# How does it fit together?



Motion  
Planning

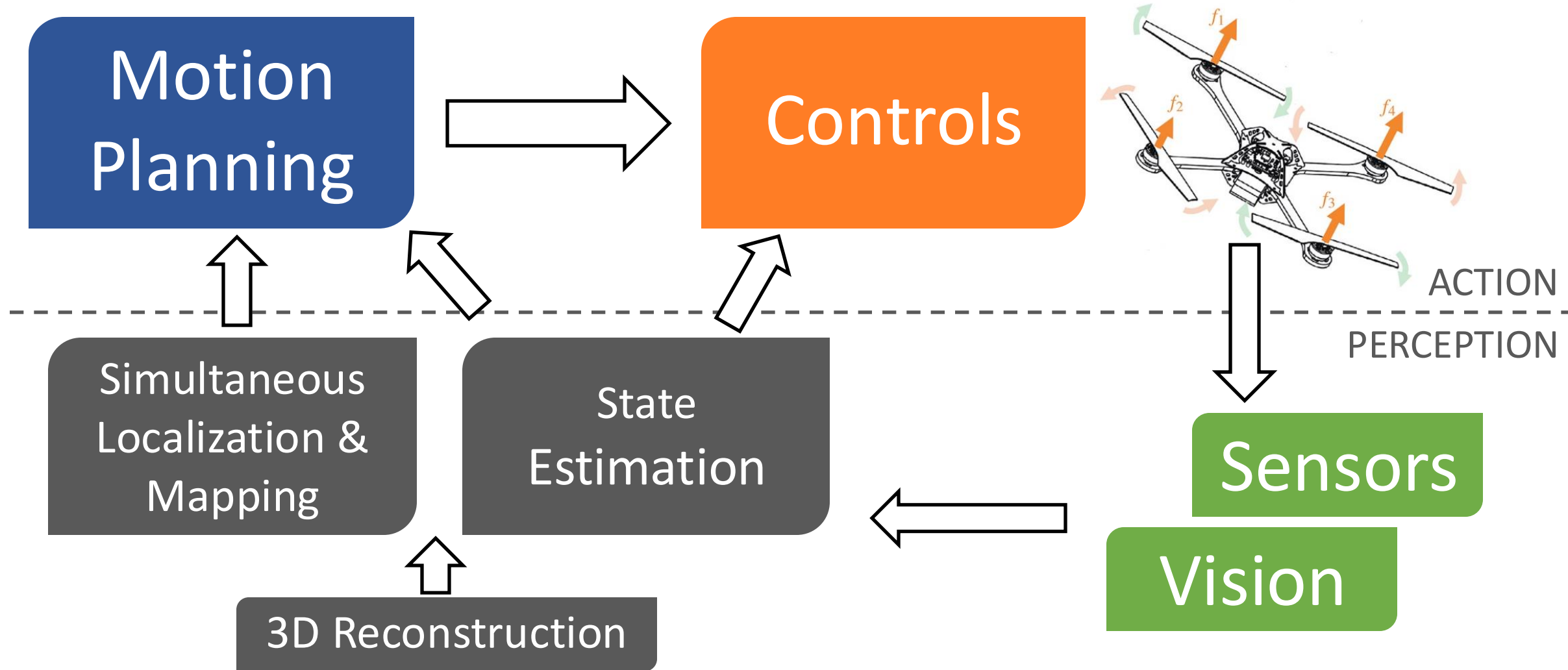


Controls

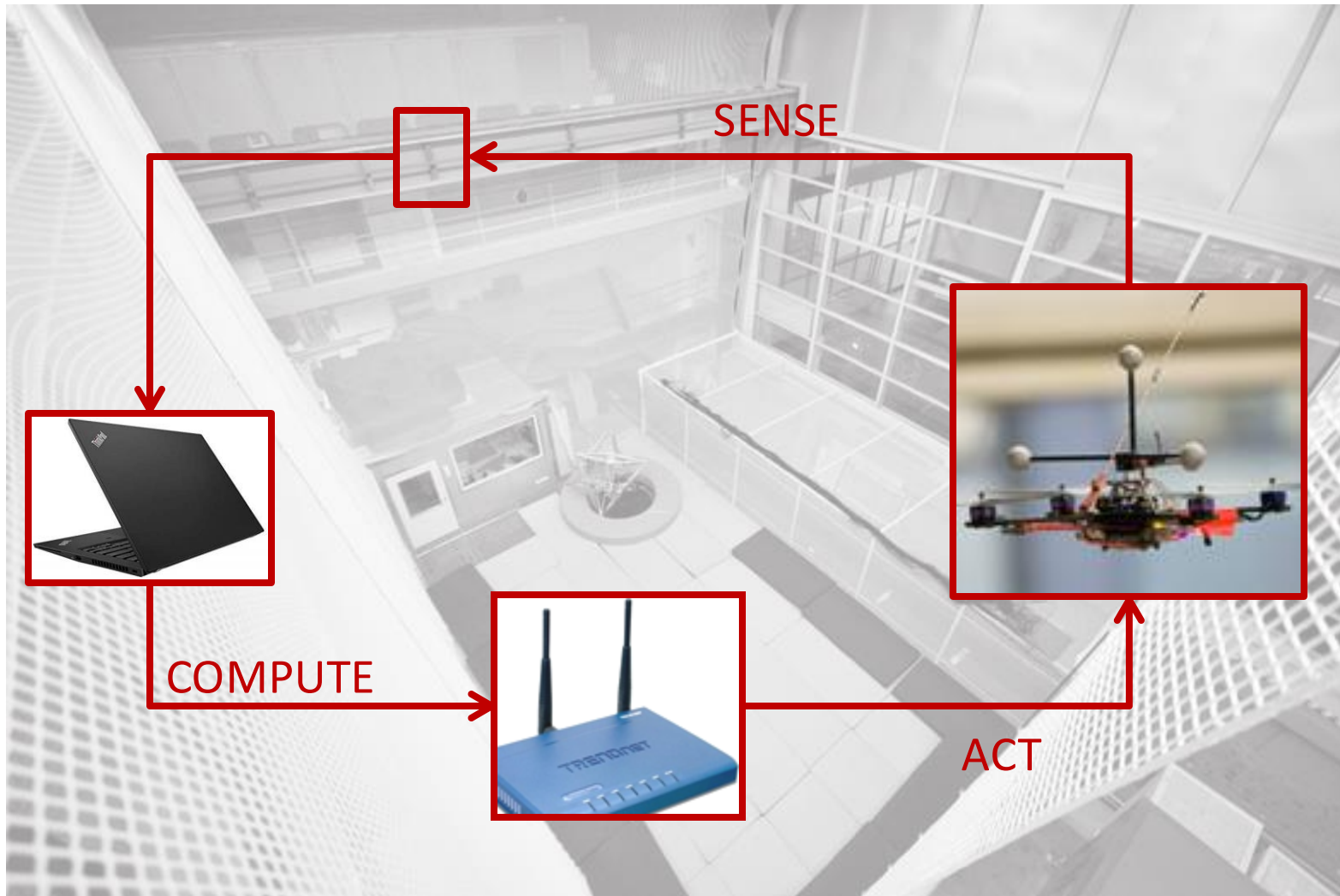


Sensors  
Vision

## How does it fit together?



# Automated Control



Physical model necessary!

$$\dot{x} = f(x(t), u(t))$$
$$y = g(x(t), u(t))$$

# Quadrotor Dynamics



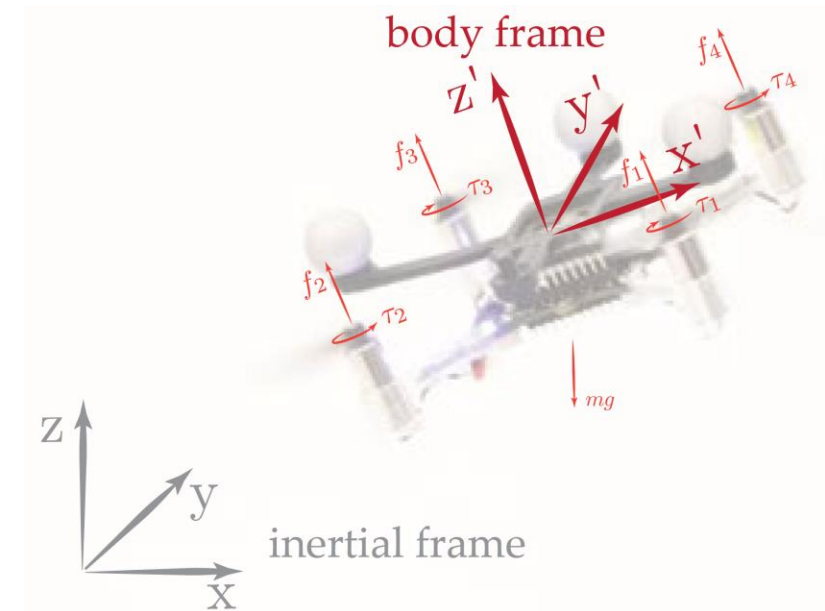
$$f_i = c_t \Omega_i^2$$



thrust constant  
drag constant

$$\tau_i = c_d \Omega_i^2 + J_m \dot{\Omega}_i$$

- The force generated by each rotor is proportional to the square of the speed
- The torque created is also proportional to the square of the speed

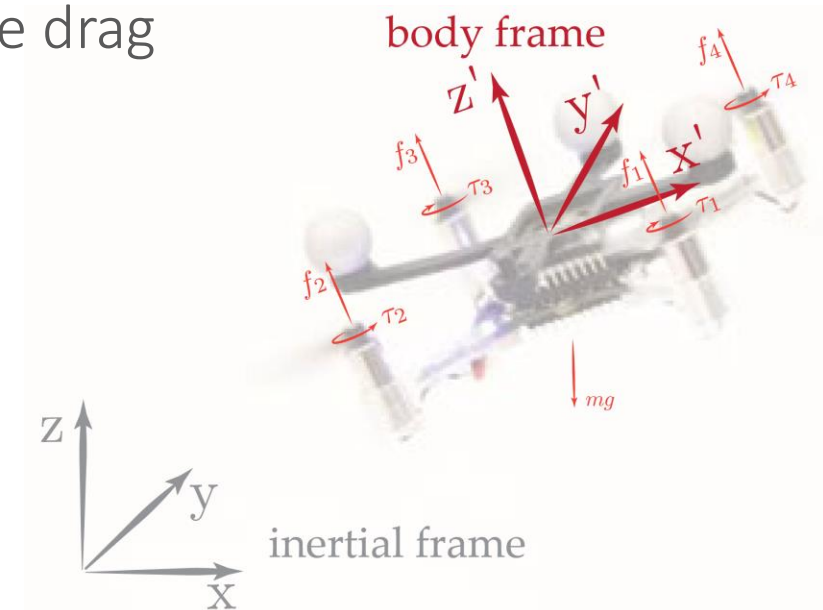




$$f_{\Sigma} = \sum_{i=1}^4 f_i$$

- The total thrust created (in the direction of the body z-axis) is the sum of all rotor forces
- The torques are acting in body frame (red)
- Torques about the x- & y-axis are from  $\tau = r \times f$
- Torque about the z-axis is from the drag

$$\begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} -l/\sqrt{2} & -l/\sqrt{2} & l/\sqrt{2} & -l/\sqrt{2} \\ -l/\sqrt{2} & l/\sqrt{2} & l/\sqrt{2} & -l/\sqrt{2} \\ -c_d/c_t & c_d/c_t & -c_d/c_t & c_d/c_t \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix}$$



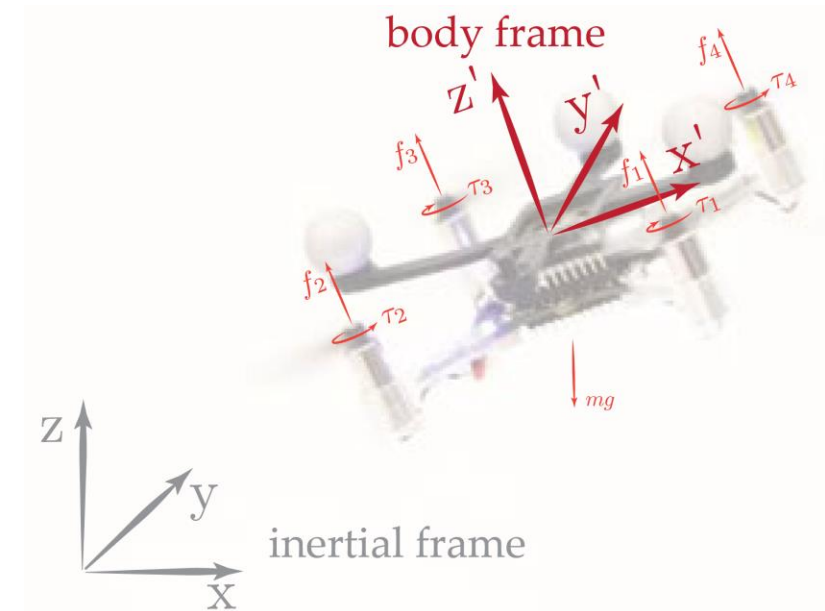


$$\dot{\mathbf{p}} = \mathbf{v}$$
$$m\dot{\mathbf{v}} = m\mathbf{g} + \mathbf{R}\mathbf{f}$$

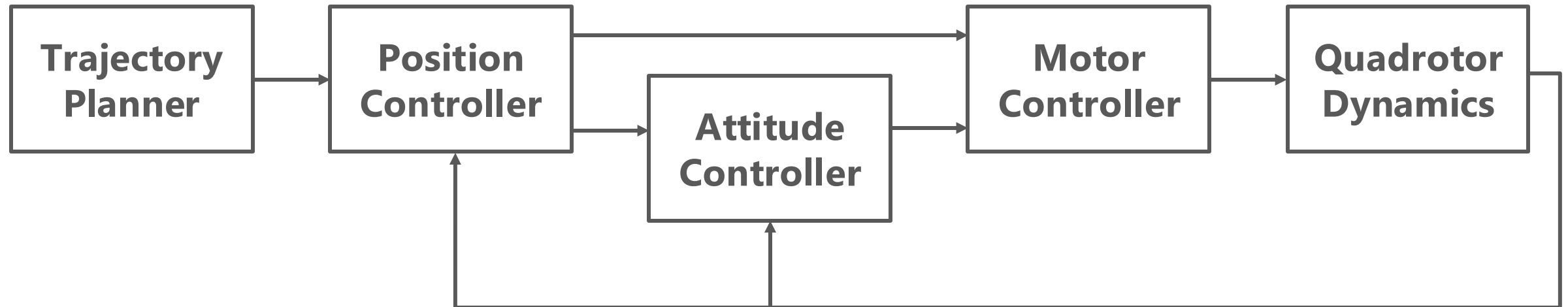
$$\mathbf{f} = \begin{bmatrix} 0 \\ 0 \\ f_{\Sigma} \end{bmatrix}$$

$$\dot{\mathbf{R}} = \mathbf{R}\mathbf{S}(\boldsymbol{\omega})$$
$$\mathbf{J}\dot{\boldsymbol{\omega}} = \mathbf{J}\boldsymbol{\omega} \times \boldsymbol{\omega} + \boldsymbol{\tau}$$

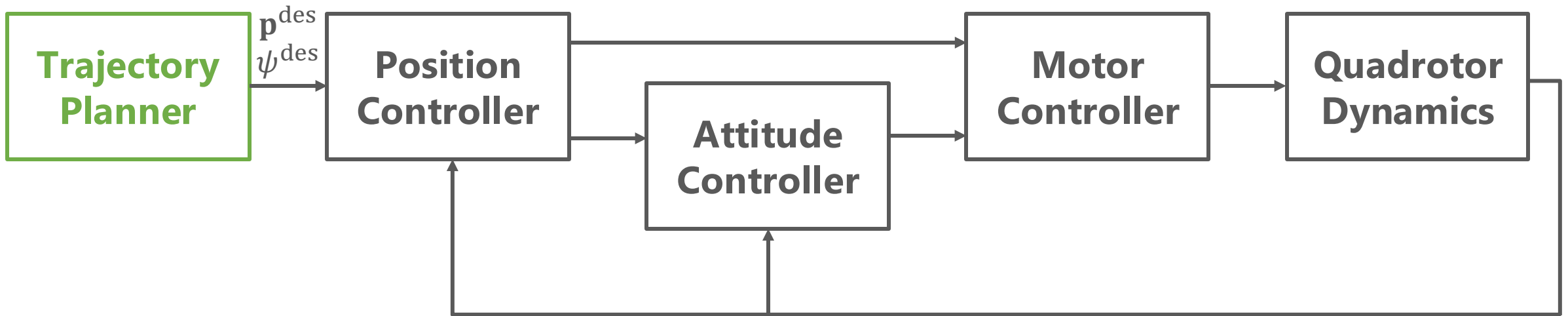
$$\boldsymbol{\tau} = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$



# Typical Planning and Control Architecture



# Typical Planning and Control Architecture



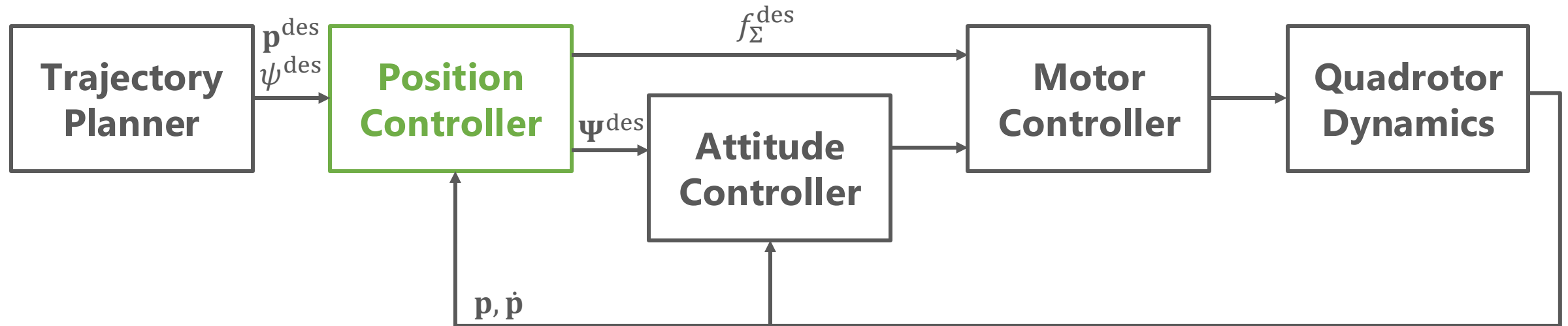
high-level objective  
(e.g., exploration, collision avoidance,  
flying through gates)



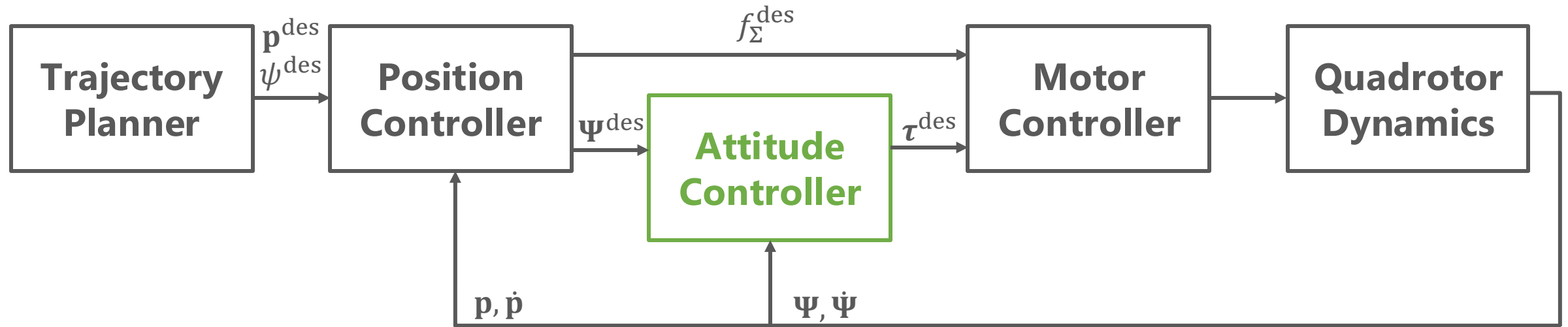
$x^{des}, y^{des}, z^{des}, \psi^{des}$   
*desired position*      *desired yaw (heading)*



# Typical Planning and Control Architecture



# Typical Planning and Control Architecture

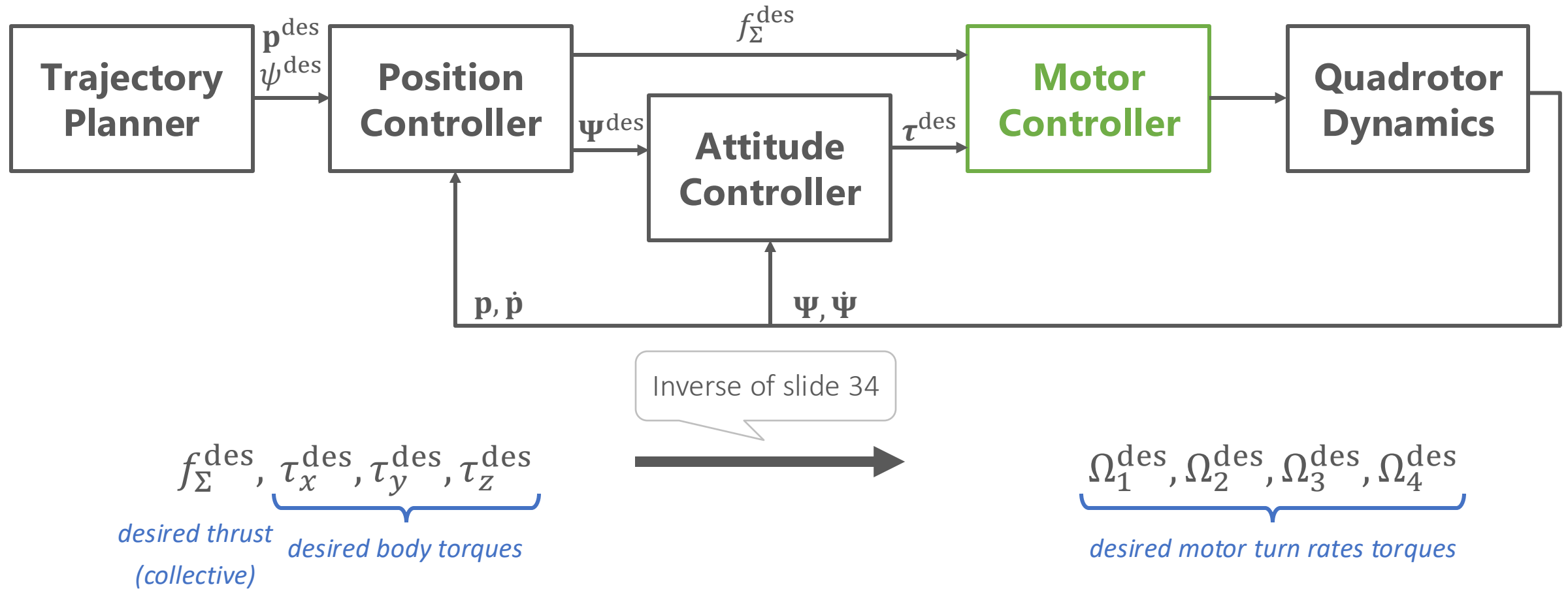


roll      pitch      yaw  
 $\phi^{\text{des}}, \theta^{\text{des}}, \psi^{\text{des}}$   
desired attitude

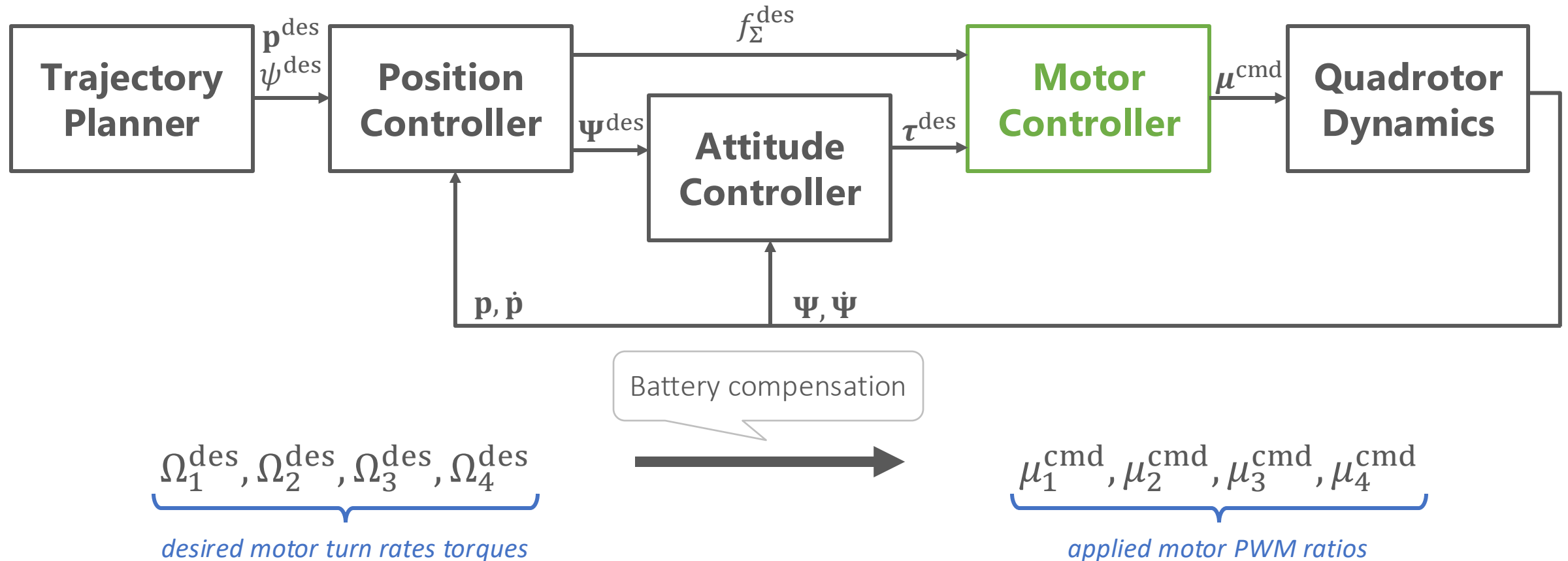


$\tau_x^{\text{des}}, \tau_y^{\text{des}}, \tau_z^{\text{des}}$   
desired body torques

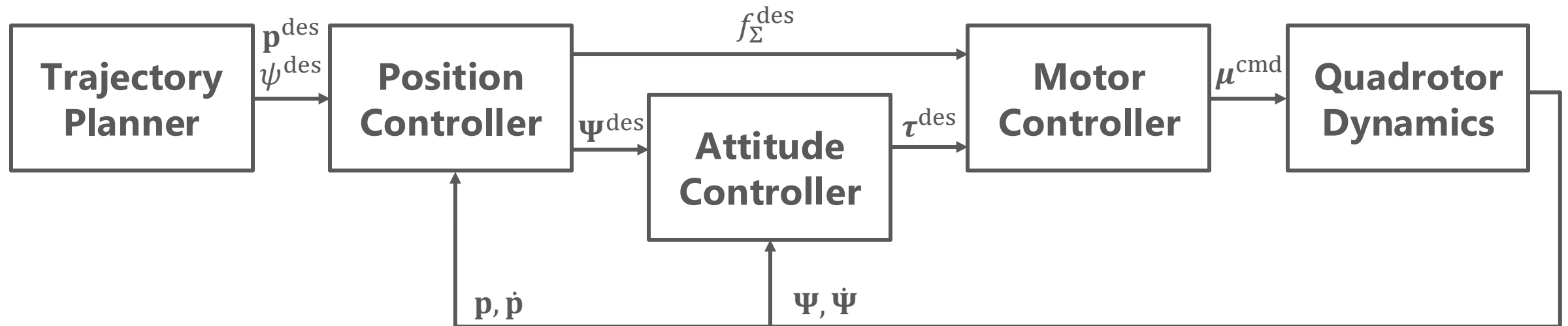
# Typical Planning and Control Architecture



# Typical Planning and Control Architecture



# Typical Planning and Control Architecture



# Quadrotor Handout



## Autonomous Drone Racing

### Quadrotor Introduction Handout

#### 1. Overview

Quadrotors are flying vehicles that have a rigid frame with four independently controlled motors, as shown in Figure 1. By changing the motor speeds, we can make the quadrotor perform different maneuvers such as moving towards a specified position or performing a flip. As compared to conventional fixed-wing flying vehicles, quadrotors are more compact and can perform vertical takeoff and landing, which makes them suitable for a wide range of applications. Some example applications include search and rescue, industrial inspection, and entertainment.

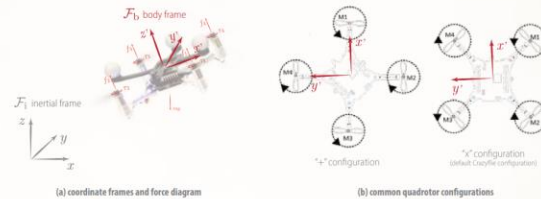


Figure 1: Illustration of a quadrotor. (a) A quadrotor is a flying vehicle equipped with a rigid frame and four motors that are controlled independently. Typically, two of the motors spin in the clockwise direction and two spin in the counterclockwise direction. Each motor generates a thrust  $f_i$  aligning with the axis of rotation and a torque  $\tau_i$  about its axis of rotation. Different quadrotor maneuvers are realized by varying the speeds of the four independently controlled motors. As an example, hovering motion can be achieved by setting the motor speeds such that  $f_i = mg/4$  for all  $i = \{1, 2, 3, 4\}$ , where  $m$  is the mass of the quadrotor and  $g$  is the acceleration due to gravitation. Generally, the translational acceleration of the quadrotor is determined by the attitude of the quadrotor, which is in turn controlled via the individual motors. There are two essential coordinate frames pertinent to every quadrotor estimation, planning, and control framework: (i) the inertial frame or the "world frame" and (ii) the body frame that is fixed to the quadrotor. The translation and rotation of the body frame relative to the world frame together characterize the pose of the quadrotor that is estimated based on sensor measurements and is used for planning and control. (b) There are two common configurations or body frame conventions for a quadrotor. The "+" configuration aligns with its  $x$  and  $y$  axes aligned with the frame, while the "x" configuration offsets by  $45^\circ$ . From a mathematical point of view, there are no major differences between the two configurations. Practically, the latter could be preferable for certain cases (e.g., mounting a front-facing camera below the frame). The Crazyflie platform used in this course follows the "x" configuration for its onboard estimation and control algorithm implementations.

#### 2. Quadrotor Dynamics

The motion of a quadrotor is fully driven by the four motors—two spinning clockwise and two spinning counterclockwise, as illustrated in Figure 1. Quadrotors are underactuated systems as it has six degrees of freedom but only four independent inputs. We briefly summarize the dynamics

of a quadrotor system in this section, which are divided into two parts: (i) motor dynamics and (ii) rigid body dynamics.

#### 2.1 Motor Dynamics

Each motor is equipped with a propeller that is firmly attached to the motor's shaft. As the motor rotates, the propeller produces a force, known as thrust  $f_i$ , that is aligned with the motor's axis of rotation, as well as a reaction torque  $\tau_i$  about the axis of rotation, due to drag with the surrounding air and change in the angular momentum. The thrust and torque produced by each motor are determined by their respective characteristic curves, which are often identified empirically. A basic motor model takes the following form:

$$f_i = c_f \Omega_i^2, \quad (1)$$

$$\tau_i = c_d \Omega_i^2 + \underbrace{I_m \dot{\Omega}_i}_{\text{change of angular momentum}}, \quad (2)$$

where  $\Omega_i$  denotes the motor rotational speed represented as revolutions per minute (RPM),  $c_f$  and  $c_d$  respectively are the lift and drag coefficients associated with the particular propeller, and  $I_m$  is the inertia coefficient. The inertia moment of the motor is often negligible; both motor thrust and torque are roughly proportional to  $\Omega_i^2$ . The lift and drag coefficients of the Crazyflie brushless are  $4.5 \cdot 10^{-10} \text{ N min}^2$  and  $2.5 \cdot 10^{-12} \text{ Nm min}^2$ , respectively.

How is the speed of the motor regulated? Without going into too much detail on brushed DC and brushless DC (BLDC) motor dynamics, a third-order relation of the applied motor voltage to the resulting thrust can be observed as

$$f_i = a v_{\text{motor},i}^3 + b v_{\text{motor},i}^2 + c v_{\text{motor},i} + d, \quad (3)$$

where the onboard motor voltage  $v_{\text{motor}}$  is pulse-width modulated (PWM) and depends on the battery voltage  $v_{\text{battery}}$  and the duty cycle  $\mu_i \in [0, 1]$  as

$$v_{\text{motor},i} = v_{\text{battery}} \cdot \mu_i. \quad (4)$$

The duty cycle, which is the proportion of the turn on to turn off time of the switch, is the control input to the system. It is easy to see that as the battery drains, i.e., a decreasing battery voltage, the input behavior changes. For that reason, onboard the Crazyflie quadrotors, there is a battery compensation implemented. This compensation works by adjusting  $\mu_i$  such that the correct  $v_{\text{motor},i}$  is applied for a given thrust command  $f_{\text{des},i}$ , according to (3) and (4).

In the following, we will assume that the battery compensation works perfectly and the actual thrust is equal to the desired thrust. Thus, we will use  $f_i$  as inputs. In reality, the motor speeds and therefore the thrust can't change instantaneously. For this introduction to quadrotor dynamics, we assume an inertia  $J_m = 0$ . However, this effect is not insignificant and should be accounted for in high-performance controllers!

#### 2.2 Rigid Body Dynamics

The motor thrusts and torques generate a collective thrust force and torque that enable different quadrotor maneuvers. The collective thrust generated by the motors is always aligned with the body  $z'$ -axis and is simply the sum of all individual forces

$$f_z = \sum_{i=1}^4 f_i. \quad (5)$$

Following the "x" configuration defined in Figure 1, we can express the collective torques in body frame as a function of the individual motor forces as

$$\boldsymbol{\tau} = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} -l/\sqrt{2} & -l/\sqrt{2} & +l/\sqrt{2} & +l/\sqrt{2} \\ -l/\sqrt{2} & +l/\sqrt{2} & +l/\sqrt{2} & -l/\sqrt{2} \\ -c_d/c_t & +c_d/c_t & -c_d/c_t & +c_d/c_t \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} \quad (6)$$

where  $l$  is the arm length of the quadrotor (i.e., the distance from the COM to the motor shaft). Note that the reaction torque generated by individual motors is opposite to their rotation direction (Figure 1).

The collective thrust  $f_z$  and the body torque  $\boldsymbol{\tau}$  are the inputs to the rigid body dynamics of the quadrotor system. A continuous-time model of the quadrotor is

$$\dot{\mathbf{p}} = \mathbf{v}, \quad (7)$$

$$m\dot{\mathbf{v}} = m\mathbf{g} + \mathbf{R}\mathbf{f}, \quad (8)$$

$$\mathbf{R} = \mathbf{R}\mathbf{S}(\boldsymbol{\omega}), \quad (9)$$

$$\mathbf{J}\dot{\boldsymbol{\omega}} = \mathbf{J}\boldsymbol{\omega} \times \boldsymbol{\omega} + \boldsymbol{\tau}, \quad (10)$$

where  $\mathbf{p} = [p_x, p_y, p_z]^T$  and  $\mathbf{v} = [v_x, v_y, v_z]^T$  are the translational position and velocity of the center of mass (COM) of the quadrotor,  $m$  is the mass of the quadrotor,  $\mathbf{R}_i$  is the rotation matrix from the body frame to the inertial frame,  $\mathbf{f} = [0, 0, f_z]^T$  is the thrust force in body frame,  $\boldsymbol{\omega} = [p, q, r]^T$  is the body angular velocity,  $\mathbf{J} = \text{diag}(J_{xx}, J_{yy}, J_{zz})$  is the inertia matrix,  $\mathbf{S}(\cdot)$  denotes the skew-symmetric matrix mapping of a vector, and  $\mathbf{g} = [0, 0, -g]^T$  is the gravitational force vector.

Note that, there are different ways of parametrizing the orientation of a quadrotor (or a rigid body in general). Euler angles and quaternions are two common parametrizations. The former is easier to interpret intuitively but has singularity issues, while the latter is easier to work with computationally and is singularity free but is unintuitive. To facilitate the controller design discussion in the following section, we briefly review how the rotational dynamics can be expressed in terms of Euler angles; one could similarly express the rotational dynamics in quaternions.

The Euler angles refer to the roll  $\phi$ , pitch  $\theta$ , and yaw  $\psi$  angles, which respectively correspond to rotations about the  $x$ ,  $y$ , and  $z$  axes. One can uniquely define a rotation matrix through these angles following a specified sequence of rotation. As an example, a Z-Y-X sequence (i.e., first rotating about the  $z$  axis by  $\psi$ , then about the  $y$  axis by  $\theta$ , and lastly about the  $x$  axis by  $\phi$ ) results in the following rotation matrix from the inertial frame to the body frame:

$$\mathbf{R}_i^b = \begin{bmatrix} \cos \theta \cos \psi & \cos \theta \sin \psi & -\sin \theta \\ \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \sin \phi \cos \theta \\ \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi & \cos \phi \cos \theta \end{bmatrix}. \quad (11)$$

Note that, the rotation matrix from the body frame to the inertial frame is  $\mathbf{R}_b^i = (\mathbf{R}_i^b)^{-1} = (\mathbf{R}_i^b)^T$ .

We can relate the body angular velocity and the time-derivatives of Euler angles as follows:



# What We Cover Today

---

1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]



# What We Cover Today

---

1. How to Set Up the Project [30 min]
2. The Online Competition [10 min]
3. Project Details and Expectations [10 min]
4. Tutorial on Quadrotor Control [20 min]
5. Next steps [10 min]





## Next Steps

---

- Beat the challenge until **April 29th**
- Lecture on algorithms
- Lecture on coding and intro to our hardware setup
- Videos on specific topics like report writing
- Start reading / exploring / coding!

# Timeline



| Week | Dates      | Content                               | Deadlines                              | Notes     | Other Dates        |
|------|------------|---------------------------------------|----------------------------------------|-----------|--------------------|
| 1    | 15/04/2026 | Lec: Kickoff                          |                                        | w/ Angela |                    |
| 2    | 22/04/2026 | Lec: Project intro and setup          |                                        | w/ Marcel |                    |
| 3    | 29/04/2026 | <i>Office Hour</i>                    | 29/04/2026 Submit to online challenge  | w/ Tutors |                    |
| 4    | 06/05/2026 | Lec: Selected algorithms for racing   |                                        | w/ Marcel |                    |
| 5    | 13/05/2026 | Lec: Hardware, code, tools + Lab tour |                                        | w/ Martin |                    |
| 6    | 20/05/2026 | <i>Lab sessions</i>                   |                                        |           |                    |
| 7    | 27/05/2026 | <i>Lab sessions</i>                   |                                        |           |                    |
| 8    | 03/06/2026 | <i>Lab sessions</i>                   |                                        |           | ICRA 01.06-05.06   |
| 9    | 10/06/2026 | <b>Progress presentations</b>         | 09/06/2026 Submit slides               |           |                    |
| 10   | 17/06/2026 | <i>Lab sessions</i>                   |                                        |           | L4DC 17.06 - 19.06 |
| 11   | 24/06/2026 | <i>Lab sessions</i>                   |                                        |           |                    |
| 12   | 01/07/2026 | <i>Lab sessions</i>                   |                                        |           | Demo Day 02.07.    |
| 13   | 08/07/2026 | <b>Final Presentations</b>            | 07/07/2026 Submit report, slides, code |           |                    |

# Questions?